Fibonacci Sequence :→ 0, 1, 1, 2, 3, 5, 8, 13, ...
0 1 2 3 4 5 6 7

n → previous

$f(0) = 0$, $f(1) = 1$

$f(2) = f(0) + f(1) = 0 + 1$

$f(3) = f(2) + f(1) = 1 + 1$

$f(n) = f(n-1) + f(n-2)$

Base Condition
if(n == 0 || n==1)
return n;

$f(7) = 13$

Recursion
A function (method) calling itself over and over until we reach the base.

fibonacci(7)
f(6) + f(5)
f(5) + f(4) + f(4) + f(3)
f(4) + f(3) + f(3) + f(2) + f(3) + f(2)
f(3) + f(2) + f(2) + f(1) + f(2) + f(1) + f(1) + f(0)
f(2) + f(1) + f(1) + f(0)
1 0

(1+1+1+1+1+1+1+1+1+1+1+1+1) = 13 main()
0+0+0+0+

Recursion Stack

fibonacci(7);
→ o/p
→

5! = 5x4x3x2x1 = 120 1! = 1 0! = 1

if (n == 0 || n == 1) {
return 1
}
return n x fact (n-1);
5 x f(4)
5 x 4 x f(3)
5 x 4 x 3 x f(2)

factorial (5) f(n) ⟹ n x f(n-1)
f(5)
→ 5 x f(4)
5 x 4 x f(3)
5 x 4 x 3 x f(2)
5 x 4 x 3 x 2 x f(1)
5 x 4 x 3 x 2 x 1 = 120

n == 0 → 1
n == 1 → 1

Recursion Stack
Sum(5) s == 0
return 5 + sum(4)
5 + 4 + sum(3)
5 + 4 + 3 + sum(2)
5 + 4 + 3 + 2 + sum(1)
5 + 4 + 3 + 2 + 1 + sum(0)
5 + 4 + 3 + 2 + 1 + 0 = 15 → base case

== 0
== 0

n == 0
→ 0

Main
Sum(5) 15

Recursion

Backtracking
{N Queens} (Choos)
(4x4) 1 Queen in each row.

(DP) Dynamic Programming
x--- y-- row-- c++

* Backtracking : Idea or Perception :
We try to input our predefined values in a normal way by simple guessing; if the guess goes wrong, we change the previous input (Backtracking)

159

(Hit & Trial Method) (Subsets)
(Sudoku) → (Recursion) Permutation
^nC_r

Power Set : The set of all subsets of an array / string is Power Set
arr = {1,2,3} n = 3 PowerSet = 2^3 = 8
(Recursion Tree)

input {1,2,3} output { }
exclude {1,2,3} { }
include {1,2,3} {1}
exclude {1,2,3} { }
include {1,2,3} {2}
exclude {1,2,3} {1}
include {1,2} {1}
exclude {1,2,3} { }
include {1,2,3} {3}
exclude {1,2,3} {2}
include {1,2,3} {2,3}
exclude {1,2,3} {1}
include {1,2,3} {1,3}
exclude {1,2,3} {1,2}
include {1,2,3} {1,2,3}
{ } 1
{3} 2
{2} 3
{2,3} 4
{1} 5
{1,3} 6
{1,2} 7
{1,2,3} 8

Array : {4,5,6} Recursion Tree (Power Set) $2^3 = 8$
Backtracking
[3]
o/p List<List<Integer>>
arr List<Integer>

{4,5,6} { }
i/p o/p
exclude {4,5,6} { }
include {4,5,6} {4}
exclude {4,5,6} { }
include {4,5,6} {5}
exclude {4,5,6} {4}
include {4,5,6} {4,5}
exclude {4,5,6} { }
include {4,5,6} {6}
exclude {4,5,6} {5}
include {4,5,6} {5,6}
exclude {4,5,6} {4}
include {4,5,6} {4,6}
exclude {4,5,6} {4,5}
include {4,5,6} {4,5,6}
{ } 1 ans
{6} 2 ans
{5} 3
{5,6} 4
{4} 5
{4,6} 6
{4,5} 7
{4,5,6} 8